PRODUCT / DESIGN / ENGINEERING

ISHPREETOS HANDBOOK

ISHPREETOS

Project Handbook and Implementation Playbook

Planning baseline for v1.0 with a controlled v1.1 backlog

Purpose

This handbook is the authoritative planning baseline for IshpreetOS. It translates the full planning conversation into product rules, interaction invariants, design direction, technical architecture, implementation order, and release criteria. It is written for Ishpreet and for future AI collaborators who need to understand the project before changing it.

Owner: Ishpreet Singh

Planning baseline: October 16, 2026

Status: Approved for implementation

Document Control

Field	Value
Document	IshpreetOS Project Handbook and Implementation Playbook
Owner	Ishpreet Singh
Baseline	October 16, 2026
Primary release	v1.0
Deferred release	v1.1
Repository	IshpreetOS - Next.js application
Execution tracker	IshpreetOS Notion board, ordered by Build Order
Authority	Approved decisions in this handbook take priority over old task wording and starter-code assumptions

How to use this handbook

- Read Product Vision, Scope, and Non-negotiable Rules before proposing architecture or UX changes.
- Use the responsive and window-behavior chapters as acceptance criteria, not as loose inspiration.
- Use the repository architecture and dependency chapters before adding files or packages.
- Use Implementation Sequence and Release Gate when selecting the next task.
- Treat items marked Prototype or Validate as deliberately unresolved. Do not silently convert them into permanent decisions.
- When a future decision changes this baseline, update both the repository documentation and the corresponding Notion task.

Rule for future AI collaborators

Do not redesign the product from first principles. First identify whether a requested change conflicts with a locked decision, explain the conflict, and ask Ishpreet before changing the governing behavior. Implementation convenience is not sufficient reason to weaken the product model.

Decision vocabulary

Label	Meaning
Locked	Approved behavior. Implement it unless Ishpreet explicitly changes the decision.
Prototype	Direction is approved, but the library or detailed interaction must pass a focused experiment.
Validate	The principle is approved; exact values require real-device or content testing.
Deferred	Intentionally excluded from v1. Do not add its packages or scaffolding.
Out of scope	Not planned. Do not infer it from the OS metaphor.

Table of Contents

Document Control 2

How to use this handbook . . . 2

Decision vocabulary . . . 2

Table of Contents 3

1. Executive Brief 7

Product statement . . . 7

The intended impression . . . 7

Success criteria . . . 7

2. Product Principles 8

Inspired, not replicated . . . 8

OS behavior with web restraint . . . 8

One source of truth . . . 8

Complexity must earn its place . . . 8

The user controls the experience . . . 8

3. Scope and Release Boundaries 9

v1.0 committed scope . . . 9

v1.1 backlog . . . 9

Explicitly excluded or deferred . . . 9

4. Information Architecture and Applications 10

Core applications . . . 10

Application instance rule . . . 10

Application registry . . . 10

No internal content maze . . . 10

5. Responsive Environment Model 11

Mode selection principle . . . 11

Client initialization . . . 11

Mobile and narrow tablet . . . 11

Spacious tablet . . . 11

Desktop . . . 11

6. Desktop and Window Interaction Model 12

Opening and cascading . . . 12

Focus and stacking . . . 12

Bounds . . . 12

Geometry by mode . . . 12

Interaction commit strategy . . . 12
Window controls . . . 12
Show Desktop . . . 12

7. State, Lifecycle, and Command Architecture 13

State ownership . . . 13
Lifecycle rules . . . 13
Reducer and commands . . . 13
Mounted but hidden applications . . . 13

8. Routing, History, Refresh, and Onboarding 14

Routing model . . . 14
Direct links . . . 14
First root visit . . . 14
Replay behavior . . . 14
Refresh semantics . . . 14
404 and crash recovery . . . 14

9. Visual Design System 15

Design direction . . . 15
Colour system . . . 15
Typography . . . 15
Window appearance . . . 15
Icons . . . 15
Wallpapers and imagery . . . 15
Density . . . 16
Motion . . . 16

10. Application Specifications 17

About . . . 17
Projects . . . 17
Experience . . . 17
Settings . . . 17
Taskbar date and time . . . 17

11. Content and Data Architecture 18

Content source layout . . . 18
Content ownership . . . 18
Validation . . . 18
Loading strategy . . . 18

12. External Integrations 19

Integration boundaries . . . 19

Notion watchlist - v1 . . . 19
Recommendation email - v1 . . . 19
Spotify - v1.1 . . . 19
Weather - v1.1 . . . 19
Caching . . . 20

13. Security, Privacy, and Analytics 21

Privacy position . . . 21
Analytics . . . 21
Cloudflare usage . . . 21
Security controls . . . 21

14. Performance, Accessibility Baseline, and Browser Support 22

Performance principles . . . 22
Accessibility baseline . . . 22
Browser support . . . 22
Physical test inventory . . . 22

15. Technical Stack and Dependency Strategy 23

Existing foundation . . . 23
Approved additions . . . 23
Dependency rules . . . 23
react-rnd prototype gate . . . 23

16. Repository and Module Architecture 24

Module responsibilities . . . 24
Provider composition . . . 25
Import rules . . . 25
Styling ownership . . . 25

17. Testing and Quality Strategy 26

Test placement . . . 26
Unit-test priorities . . . 26
Component-test priorities . . . 26
Playwright journeys . . . 26
Coverage philosophy . . . 26
Universal phase gate . . . 27

18. Implementation Sequence 27

Application order . . . 27
Content parallel track . . . 27
Timeline policy . . . 28

19. Development Workflow and Ownership 28
Learning-first ownership . . . 28
Branch workflow . . . 28
Planning and execution sources . . . 28
20. v1 Release Gate 29
21. Non-negotiable Invariants 30
22. Prototype and Validation Register 31
23. Operating Instructions for Future AI Collaborators 32
Before changing code . . . 32
While implementing . . . 32
When a decision appears inconsistent . . . 32
When finishing a task . . . 32
Appendix A. Window Transition Reference 33
Appendix B. Notion Build Order 34
Appendix C. Technical Reference Links 35
Appendix D. Glossary 36

1. Executive Brief

IshpreetOS is a portfolio presented as a personal operating system. It should feel recognizably desktop-like without becoming a gimmick or a replica of Windows, macOS, or Linux.

Product statement

IshpreetOS is a retro-modern web portfolio that lets visitors explore Ishpreet's biography, work experience, projects, settings, watchlist, and recommendation form through an operating-system interaction model. The site should feel deliberate, personal, fast, and technically credible. The OS metaphor is the organizing interface, not an excuse to hide information or make normal web behavior difficult.

The intended impression

- Personal: the operating system, wallpapers, photography, content, and visual voice should clearly belong to Ishpreet.
- Technical: window management, responsive transitions, routing, state, validation, and testing should demonstrate engineering care.
- Readable: the portfolio content remains the priority; retro styling never makes paragraphs harder to consume.
- Playful but controlled: the desktop metaphor invites exploration without excessive animation, sound, or novelty controls.
- Shareable: every meaningful application has a real URL and useful metadata.
- Fast: local content and the desktop appear without waiting for Notion or any other external service.

Success criteria

- A visitor understands how to explore the site without instructions longer than the welcome message.
- Direct links behave like normal web pages while still opening the correct IshpreetOS application.
- The experience is coherent on an iPhone, iPad, desktop monitor, and resized browser window.
- Application state feels OS-like within a session, while refresh behavior stays predictable and privacy-friendly.
- External integrations enhance the portfolio but can fail without taking down the desktop.
- The repository remains understandable to Ishpreet after time away from the project.

2. Product Principles

Inspired, not replicated

The project may reference familiar desktop conventions - cascading windows, taskbar switching, traffic-light controls, layered windows, and a terminal-style welcome - but it must not reproduce another operating system. The name, palette, application structure, wallpapers, interaction details, and written voice should create IshpreetOS as its own product.

OS behavior with web restraint

- Use OS conventions where they make state and navigation easier to understand.
- Keep browser-native expectations such as real URLs, Back and Forward, safe external links, responsive content, and semantic controls.
- Do not implement desktop features only because real operating systems have them.
- Prefer a small number of deeply finished behaviors over a large number of novelty features.

One source of truth

- Application definitions live in the application registry.
- Window transitions live in the reducer and command layer.
- Routes and metadata derive from registry data.
- Portfolio content lives in validated Markdown and YAML.
- Persistent preferences live in the preferences system, not window state.
- The Notion board tracks execution; repository documents explain the implementation model.

Complexity must earn its place

Every dependency, abstraction, animation, and integration must remove meaningful risk or complexity. Experiments that fail are removed fully. Packages are added when their phase begins, not because they may be useful later. There is no reward for keeping unused scaffolding in the repository.

The user controls the experience

- No automatic sound.
- No surprise location request.
- No custom colour picker capable of breaking readability.
- No visitor account or server-side preference profile.
- No forced onboarding for a shared direct link.
- Motion follows the operating system by default and can be reduced explicitly.

3. Scope and Release Boundaries

v1.0 committed scope

- Responsive IshpreetOS desktop shell, wallpaper, fixed desktop icon grid, and persistent taskbar.
- Single-instance application model with open, focus, minimize, maximize, restore, close, and stacking behavior where the active mode permits it.
- About, Projects, Experience, and Settings applications.
- First-visit boot sequence and one-time terminal-style welcome.
- Dedicated application routes, browser-history coordination, SEO metadata, sitemap, robots rules, and social previews.
- Validated local Markdown and YAML content.
- Read-only Notion watchlist inside About.
- Protected recommendation form delivered through email, with no submission database.
- Approved appearance preferences, time/date formatting, motion control, privacy, credits, and welcome replay.
- Anonymous, meaningful analytics and performance monitoring through Vercel.
- Error boundaries, OS-themed recovery, security headers, licence tracking, and cross-device tests.

v1.1 backlog

- Spotify-only music widget. No Last.fm fallback.
- Toronto weather widget powered by Open-Meteo with a one-hour server cache.
- Optional terminal Easter eggs after the basic welcome terminal has proven useful.

No Spotify, weather, or terminal-Easter-egg package, credential, or implementation scaffold belongs in v1. The adapter ideas may be documented, but the code is deferred.

Explicitly excluded or deferred

- True dark mode. Dark wallpapers and accents are allowed, but application windows remain cream in v1.
- Custom accent colour picker.
- User-uploaded wallpapers.
- Movable desktop icons.
- Multiple instances of the same application.
- Window resizing on touch-first tablet modes.
- Mobile or narrow-tablet window dragging.
- Progressive Web App installation and offline support.
- General contact form. Direct contact remains email and social links; the form is recommendations only.
- Visitor accounts, cloud-synced settings, or a recommendation database.
- Audio, boot sounds, click sounds, or autoplaying music.
- Sentry and Cloudflare Web Analytics at v1 launch.
- Full advanced accessibility programme during initial development. Baseline semantics, keyboard behavior, focus, contrast, and reduced motion are still required.

4. Information Architecture and Applications

Core applications

Application	Route	Purpose	Primary content
About	/about	Personal introduction and visitor connection	Biography, reactive heading, social links, resume, watchlist, recommendation form
Projects	/projects	Portfolio work	Project cards, status, screenshots, technologies, GitHub, optional live site
Experience	/experience	Professional history	Timeline entries with role, employer, dates, location, responsibilities, technologies
Settings	/settings	Local appearance and system preferences	Wallpaper, accent, clock/date, motion, resets, privacy, credits
Privacy	/privacy	Direct route into Settings privacy content	Analytics, recommendation processing, service providers, no ad tracking

Application instance rule

There is exactly one instance of each application. Activating a desktop icon for an already-open application restores and focuses that existing instance. It never resets the application, opens another copy, or changes its original taskbar position.

Application registry

- The registry owns application ID, route, display title, icon key, desktop order, metadata, and whether an app appears on the desktop.
- The component loader map is separate so static metadata does not depend on React implementations.
- The reducer knows application IDs and state only. It does not know routes or component imports.
- Route aliases, including /privacy, resolve through registry/navigation configuration.

No internal content maze

About is one continuous scrolling page. It does not use tabs or carousels. Projects and Experience remain straightforward content applications. Settings may group controls into clear sections, but navigation should not turn into a second miniature operating system inside the window.

5. Responsive Environment Model

Mode selection principle

Interaction capability is based on usable desktop space, orientation, viewport height, and pointer capability - not on marketing labels for hardware. A large foldable may behave like a tablet; a short coarse-pointer landscape layout remains mobile even if its width is large.

Mode	Initial heuristic	Visible windows	Move	Resize	Controls
Mobile	Under roughly 640 px, or short coarse-pointer landscape	One	No	No	Minimize and close
Narrow tablet	Roughly 640-899 px without enough usable area	One	No	No	Minimize and close
Spacious tablet	Roughly 900 px and enough usable height, usually touch	Multiple	Yes	No	Minimize, maximize/restore, close
Desktop	Roughly 1100 px plus fine pointer and sufficient height	Multiple	Yes	Yes	Minimize, maximize/restore, close

These numeric breakpoints are starting heuristics, not sacred constants. Final thresholds must be tested against usable area, actual window proportions, taskbar height, the iPhone 15 Pro, the iPad Air, foldable emulation, and resized desktop browsers.

Client initialization

- The server renders the wallpaper, desktop shell, and taskbar structure.
- Windows are not instantiated until the client has measured the environment and chosen a mode.
- The delay should be nearly imperceptible and must not use the artistic boot screen as cover.
- One centralized responsive controller owns mode selection. Components consume the result instead of inventing their own breakpoints.
- Resize, rotation, and virtual-keyboard changes trigger safe recalculation.

Mobile and narrow tablet

- Only one open application is visible at a time; other open applications stay mounted and appear in the taskbar.
- Selecting another taskbar application hides the current one and displays the selected one.
- The window is a fixed inset surface that fills the usable desktop without covering the taskbar.
- The window is not movable or resizable.
- Rotation recomputes the inset rather than preserving portrait geometry.
- The taskbar may collapse nonessential information while the virtual keyboard is visible.

Spacious tablet

- Multiple windows may be visible and stacked.
- Windows can be freely dragged within the usable desktop.
- Free resizing is disabled because precision resizing without a cursor is a poor touch experience.
- Focus, minimize, maximize/restore, close, and stacking remain available.

Desktop

- Multiple windows may be visible, dragged, resized, minimized, maximized, restored, focused, closed, and stacked.
- Maximize fills the usable desktop only. It does not cover the IshpreetOS taskbar, browser chrome, or the visitor's operating-system UI.
- Browser resizing remains smooth and clamps geometry only when required.

6. Desktop and Window Interaction Model

Opening and cascading

- The first window opens near the center of the usable desktop.
- Each newly opened window appears above the previous one, offset approximately 24 px down and right.
- The cascade wraps when another offset would leave the usable desktop.
- Applications do not declare different default positions.
- When mode-specific geometry does not exist, the cascade algorithm initializes it.

Focus and stacking

- Focusing a window brings it to the front without changing its x/y position.
- It may cover other windows completely; that is acceptable.
- Focus changes stacking order only. They never reorder taskbar icons.
- A click on an inactive window both focuses it and activates the clicked control. Visitors do not need a first click only to focus.
- Application and desktop icons use single-click or single-tap activation everywhere.

Bounds

No part of a window may leave the usable desktop. The title bar, edges, and controls always remain inside the available area. Dragging, resizing, restoring old geometry, orientation changes, and viewport shrinkage all pass through the same bounds rules.

Geometry by mode

- Desktop stores position and size.
- Spacious tablet stores its own position while using mode-controlled size.
- Mobile and narrow tablet store no window geometry.
- Changing modes selects the target mode's remembered geometry without overwriting another mode's values.
- Geometry is session state only. It is not written to local storage.

Interaction commit strategy

During live drag or resize, temporary geometry belongs to the interaction layer. The reducer receives the final committed geometry at the end of the interaction. This avoids flooding global state, reduces unnecessary renders, and keeps the reducer's history meaningful.

Window controls

Control	Colour	Behavior
Maximize / restore	Green	Toggle between usable-desktop maximum and remembered normal geometry where the mode permits it
Minimize	Yellow	Hide the application while preserving mounted state and taskbar presence
Close	Red	Close immediately, remove from taskbar, unmount content, and destroy transient state

The order is green, yellow, red from left to right on the right side of the title bar. Closing is intentionally immediate. There is no unsaved-form warning; accidental closure is accepted behavior.

Show Desktop

- The permanent IshpreetOS taskbar button minimizes every open window.
- It does not close applications or reset their state.
- It navigates to the root route.
- Browser Back returns to the previously routed application and restores/focuses it through the routing coordinator.

7. State, Lifecycle, and Command Architecture

State ownership

State	Owner	Persistence
Open/minimized/maximized status	Desktop session reducer	Current browser session only
Taskbar order	Desktop session reducer	Current browser session only
Stacking order	Desktop session reducer	Current browser session only
Mode-specific geometry	Desktop session reducer	Current browser session only
Live drag/resize geometry	WindowInteractionLayer	Only during interaction
Application form/scroll/component state	Mounted application component	Until close or refresh
Wallpaper, accent, clock/date, motion	Preferences provider	Local storage for that browser
Onboarding completion	Preferences/onboarding store	Local storage for that browser
Route	Next.js and routing coordinator	Browser history

Lifecycle rules

Event	Application state	Component state	Taskbar
Open closed app	Open and focused	Newly mounted	Append to right
Activate open app	Restore if needed and focus	Preserved	No reorder
Minimize	Open and minimized	Mounted and preserved	Remains
Mobile switch	Previous hidden; selected visible	Both preserved	No reorder
Show Desktop	All open apps minimized	Preserved	No reorder
Close	Removed	Unmounted and destroyed	Removed
Close then reopen	New open session	Fresh state	Append to right
Refresh	All apps closed	Destroyed	Cleared

Reducer and commands

- The reducer manages desktop session state only.
- Routing stays outside the reducer.
- Applications call an OS command API rather than dispatching internal action shapes.
- Pure actions, reducer, selectors, cascade, geometry, and bounds functions contain no React code.
- A small development-only logger records commands and resulting state. Production removes it.
- Separate state and action contexts limit unnecessary subscription to changing state.

Mounted but hidden applications

Minimized and mode-hidden applications remain mounted so unfinished form data, scroll positions, and component state survive. They receive a visibility signal so timers, animation, and future Spotify/weather polling can pause while hidden.

8. Routing, History, Refresh, and Onboarding

Routing model

- The root route represents the bare desktop.
- Application URLs use a registry-driven dynamic route while remaining real, shareable paths.
- A meaningful app opening pushes history.
- A simple focus change uses replace rather than adding history noise.
- The routing coordinator maps route changes into OS commands and maps meaningful OS navigation into routes.
- The reducer never imports or operates the Next.js router.

Direct links

A visitor entering /projects, /about, /experience, /settings, or /privacy goes directly to the desktop with the requested application open. The boot sequence and welcome terminal are bypassed, and onboarding is marked complete. This prioritizes sharing and search intent over theatrical onboarding.

First root visit

1. Render the desktop shell and complete client environment measurement.
2. Show a brief artistic 'Loading IshpreetOS' boot sequence that is not tied to real network loading.
3. Enter the desktop.
4. Open the concise terminal-style welcome window.
5. When the welcome closes, leave the visitor on the normal desktop.
6. Persist onboarding completion locally so normal return visits skip both pieces.

Replay behavior

Settings includes Replay Welcome as an explicit action. Replaying onboarding does not reset appearance settings. Reset Appearance does not replay onboarding. These are separate commands because they represent different visitor intent.

Refresh semantics

- Close every open application.
- Destroy unfinished recommendation form input and all other transient app state.
- Clear taskbar and window geometry.
- Preserve preferences and onboarding completion.
- On a direct application URL, rebuild a fresh session with that application open.

404 and crash recovery

- Unknown routes use a themed IshpreetOS application-not-found presentation while retaining correct HTTP 404 semantics.
- Serious rendering failures use a compact recovery screen rather than a conventional unrelated error page.
- Production error UI never exposes stack traces, tokens, or raw provider responses.

9. Visual Design System

Design direction

The visual language is visibly retro and OS-inspired, but the content interiors remain modern and spacious enough for a portfolio. The browser should feel like Ishpreet's operating system, not like a reskinned copy of a historic desktop.

Colour system

Role	Decision
Application surface	Warm off-white cream; never pure white
Body text	Pure black for strong readability
Default accent	XP-inspired blue
Approved accents	Blue, green, purple, orange, and pink
Destructive/control red	Reserved for close, errors, and fixed status semantics
Alternative warm signal	Pink may replace red when a decorative accent is needed
Inactive treatment	Subtle gray veil, softened border, and muted shadow
Active treatment	Accent border, stronger shadow, and no gray veil

Exact colour values require prototype testing for contrast across wallpapers. Visitors choose only from pre-approved accents so the site cannot be configured into an unreadable state.

Typography

- Desktop and OS chrome use an easy-to-read pixel-style font.
- Application headings may use the pixel font selectively.
- Application body content uses a Times-like open-source serif, self-hosted through next/font/local.
- Terminal content uses an authentic terminal-style monospace font.
- Long paragraphs never receive reactive distortion or pixel typography.
- All font files are self-hosted for consistent rendering and fewer external requests.

Window appearance

- Rounded retro-modern shape inspired by compact macOS-style windows.
- Cream title bar continues into the cream application surface instead of becoming a solid blue strip.
- Accent border communicates customization and active state.
- Application icon and pixel title appear on the left; controls appear on the right.
- Title bar remains fixed while the application content region scrolls.

Icons

- Desktop application icons come from one cohesive classic pixel-art computer icon family.
- Do not mix multiple pixel-art families.
- Lucide remains available for small internal controls where a pixel icon would reduce clarity.
- Every icon and pack must be publicly licensed for the intended use and listed in credits.

Wallpapers and imagery

- Use nostalgic landscapes, abstract XP-style imagery, and selected personal photography.
- Do not use minimalist pattern wallpapers.
- No visitor wallpaper upload in v1.

- Store all selectable wallpapers together, regardless of whether they are third-party art or personal photography.
- Strip EXIF and location metadata from personal photographs.
- Record focal-point metadata so responsive cropping preserves the subject.

Density

The desktop should feel like a real computer without becoming crowded. Application interiors use medium density: enough whitespace for a modern portfolio, enough visible structure to preserve the OS illusion, and no oversized marketing-site spacing that wastes the window.

Motion

- Motion is restrained and functional, never bouncy or toy-like.
- CSS handles hover, focus, colour, shadow, tooltip, and wallpaper fades.
- Motion may be added only after window interaction is stable, for open, close, minimize, restore, and maximize transitions.
- Live dragging and resizing do not use the animation library.
- Wallpaper changes crossfade rather than flashing.
- System is the default motion setting; visitors may explicitly choose Standard or Reduced.

10. Application Specifications

About

- One continuous scrolling document with no tabs or carousel.
- A reactive heading or short phrase may move around the pointer; paragraphs remain stable.
- Biography content comes from validated Markdown and uses the serif body font.
- Social links, email, and resume are explicit actions.
- The watchlist appears as a simple list: title, media type, and status only.
- The recommendation form follows the watchlist in the natural document flow.
- No general-purpose contact form is added.
- Minimizing preserves scroll and unfinished input; closing destroys both immediately.

Projects

- Each project comes from its own Markdown file.
- Cards include name, description, status, completion date where applicable, technologies, and screenshot.
- The entire card is not clickable.
- Use explicit GitHub and optional Live Site buttons.
- Completed and in-progress states use approved treatments and must not rely only on colour.
- Technology tags are a prototype detail; simplify them if they make cards noisy.

Experience

- Each position comes from its own Markdown file.
- Entries include company, role, location, start/end dates, responsibilities, and technologies.
- Spacious layouts use a vertical timeline showing when each position began and ended.
- Mobile uses a straightforward stacked presentation.
- The timeline should communicate chronology without turning into a decorative infographic.

Settings

- Wallpaper preset selection.
- Approved accent selection.
- 12-hour or 24-hour clock.
- Automatic, month-first, or day-first date format.
- System, Standard, or Reduced motion.
- Reset Appearance and Replay Welcome as separate actions.
- Privacy explanation and asset credits.
- No true dark mode, custom colour picker, font picker, or uploaded wallpaper.

Taskbar date and time

- Use the visitor's local time; this requires no location permission or cookies.
- Do not display seconds.
- Date formatting follows the stored preference.
- When space is constrained, abbreviate or hide the date before harming app switching.

11. Content and Data Architecture

Content source layout

```
content/
|-- profile.md
|-- site.yaml
|-- credits.yaml
|-- experience/
|   |-- position-one.md
|   `-- position-two.md
`-- projects/
    |-- ishpreetos.md
    `-- another-project.md
```

Content ownership

Source	Owns
site.yaml	Site title, description, public location, email, social links, resume URL, and default SEO fields
profile.md	Biography heading, short introduction, and long-form personal content
experience/*.md	One validated professional position per file
projects/*.md	One validated project per file
credits.yaml	Asset name, creator, source, licence, usage, and required attribution
Notion data source	Watchlist title, media type, and status only

Validation

- Zod is the single validation system for content, environment variables, forms, settings loaded from storage, and external API payloads.
- Types are inferred from schemas instead of being repeated manually.
- Invalid local content fails loudly during development and build.
- Malformed external data becomes a controlled unavailable result rather than crashing the desktop.
- Markdown uses gray-matter, react-markdown, remark-gfm, and no raw HTML or MDX.

Loading strategy

- All small local content loads and validates on the server when the desktop is prepared.
- Typed serializable view data is passed into application renderers.
- Local content does not require a browser fetch for each app opening.
- Application UI code may still load separately.
- External integrations remain lazy and cannot block desktop display.

12. External Integrations

Integration boundaries

Server integrations are organized by capability, not provider. The rest of IshpreetOS asks for a watchlist or sends a recommendation; it does not depend directly on Notion, Resend, Turnstile, or Upstash APIs. Provider-specific adapters remain replaceable.

Notion watchlist - v1

- Use a read-only internal integration shared only with the watchlist data source.
- Keep the token server-only.
- Map provider responses into a local WatchItem schema.
- Show title, media type, and planned/in-progress/completed status only.
- No poster, artwork, rating, synopsis, or external information link.
- Cache for six hours.
- Keep freshness invisible unless the integration is unavailable.
- Use cached data when possible; otherwise show a compact retryable unavailable state.

Recommendation email - v1

Field	Requirement
Name	Optional
Email	Optional
Media type	Required: movie, television show, or book
Media name	Required

- Use a Next.js Server Action and typed action result.
- Validate with Zod on the server.
- Protect with a honeypot, realistic minimum completion time, Turnstile verification, and Upstash rate limiting.
- Deliver through Resend to a server-configured personal inbox.
- Do not create a submission database or automatic confirmation email.
- Local development logs a sanitized preview; previews use test keys and test recipients; only production reaches the real inbox.
- Never place form contents, email addresses, or names in analytics or routine logs.

Spotify - v1.1

- Spotify only; do not revive the Last.fm fallback.
- Use server-side authorization-code flow for private user data.
- Keep credentials and refresh tokens server-only.
- Plan for periodic manual reauthorization.
- Poll only while visible and use a short shared server cache.
- Revisit the interface and design before implementation because it is intentionally deferred.

Weather - v1.1

- Use Open-Meteo through standard server-side fetch.
- Use fixed Toronto coordinates.
- Never request visitor geolocation.
- Cache for one hour.

- Use locally controlled weather icons.
- A provider failure affects the widget only.

Caching

Resource	Server cache	Client behavior
Notion watchlist	6 hours	No recurring client polling
Toronto weather	1 hour	Check on relevant widget open and occasionally while visible
Spotify playback	Approximately 15-30 seconds	Poll the IshpreetOS route every 30-60 seconds only while visible
Spotify recent history	Several minutes	Refresh on open or focus

Next.js server caching plus small purpose-built hooks is sufficient. Do not add SWR or TanStack Query unless later requirements introduce many live resources, mutations, optimistic updates, authentication, or offline behavior.

13. Security, Privacy, and Analytics

Privacy position

- No visitor account.
- No advertising or cross-site tracking.
- No cookie-based settings; local storage is sufficient.
- No recommendation-submission database.
- No location request for time or weather.
- The Settings privacy section explains analytics, optional submitted contact data, service providers, and processing behavior in plain language.

Analytics

- Use Vercel Web Analytics for anonymous page and approved custom events.
- Use Vercel Speed Insights for Core Web Vitals.
- Track meaningful events only: application opens, external project links, and submission success/failure categories where no personal data is attached.
- Do not track raw form data, email addresses, names, window geometry, pointer paths, or every focus change.
- Do not add Cloudflare Web Analytics at launch because it duplicates Vercel and does not provide the needed custom-event model.
- Do not add Sentry until production diagnosis demonstrates a need.

Cloudflare usage

Cloudflare Turnstile is approved for the recommendation form and works independently of Cloudflare proxying. The full Cloudflare reverse proxy/CDN should not be placed in front of Vercel because it adds overlapping caching, reduces platform traffic visibility, and complicates reliability. Cloudflare DNS can be evaluated separately during deployment if the domain already uses it.

Security controls

- Zod validation at every untrusted boundary.
- Server-only modules for credentials and provider clients.
- Turnstile token verification on the server.
- Upstash rate limiting for the recommendation action.
- Honeypot and completion-time checks as additional bot signals.
- Strict Content Security Policy with explicit provider allowances.
- HSTS, nosniff, Referrer Policy, and a restrained Permissions Policy.
- noopener and appropriate noreferrer behavior for external tabs.
- No raw HTML in Markdown.

14. Performance, Accessibility Baseline, and Browser Support

Performance principles

- Render the desktop before slow external integrations.
- Avoid layout shifts as apps or integrations load.
- Use responsive images and optimize personal photography.
- Keep drag and resize updates local to the interaction layer.
- Pause hidden app effects and future widget polling.
- Use code splitting for application UI where it provides measurable value.
- Remove visual elements or dependencies that cannot meet the performance target.
- Audit the production bundle and unused dependencies before release.

Accessibility baseline

A comprehensive accessibility programme is deferred, but basic quality is not optional. Every interactive element uses a semantic button or link, visible focus, sufficient contrast, understandable labels or tooltips, and keyboard-reachable controls. Reduced motion is supported from the beginning. Status never relies on colour alone.

Browser support

- Current Chrome and Edge on desktop.
- Current Firefox on desktop.
- Current Safari on iPhone and iPad.
- Current mainstream Android Chromium through emulation and a real-device smoke test when available.
- Internet Explorer and obsolete browsers are not supported.

Physical test inventory

- Windows PC with two 27-inch 1440p displays at 165 Hz and 144 Hz.
- iPhone 15 Pro for real mobile Safari, safe areas, landscape, and virtual keyboard behavior.
- iPad Air fifth generation with M1 for real touch and tablet layouts.
- Playwright emulation for Android, folding-phone dimensions, and lower-resolution laptops.

15. Technical Stack and Dependency Strategy

Existing foundation

Area	Choice
Framework	Next.js 16 App Router
UI runtime	React 19
Language	TypeScript in strict mode
Styling	Tailwind CSS 4, CSS variables, narrow CSS Modules
Primitives	shadcn/Base UI
Internal icons	Lucide
Unit/component tests	Vitest and Testing Library
End-to-end tests	Playwright
Hosting	Vercel
Package manager	pnpm

Approved additions

Dependency	When	Purpose
zod	Foundation	Runtime schemas and inferred types
gray-matter	Content foundation	Markdown frontmatter
react-markdown	Content foundation	Safe Markdown rendering
remark-gfm	Content foundation	Tables and task lists
yaml	Content foundation	site.yaml and credits.yaml
react-rnd	Only after prototype passes	Desktop drag/resize base behind WindowInteractionLayer
Motion	After stable window behavior	Window lifecycle and layout transitions
Resend	Integration phase	Recommendation email delivery
Upstash rate-limit/Redis	Integration phase	Shared serverless rate limiting
Provider-specific Notion support	Integration phase	Read-only watchlist adapter; prefer minimal surface

Dependency rules

- Add packages just in time for the phase that uses them.
- Wrap replaceable third-party libraries behind project-owned modules.
- Prefer platform APIs and standard fetch when a provider SDK offers little value.
- Remove a failed experiment fully, including package, config, types, and dead adapter code.
- Do not retain libraries for hypothetical future work.
- Do not add Redux, Zustand, SWR, TanStack Query, a CSS-in-JS runtime, or a global animation framework without a new demonstrated need.

react-rnd prototype gate

- Mouse drag and resize.
- Touch drag on iPad.
- Strict bounds and taskbar exclusion.
- Controlled temporary geometry with final commit.
- Maximize/restore and responsive-mode transitions.
- Multiple overlapping windows at acceptable frame rate.

16. Repository and Module Architecture

```
IshpreetOS/
|-- content/
|   |-- profile.md
|   |-- site.yaml
|   |-- credits.yaml
|   |-- experience/
|   `-- projects/
|-- docs/
|   |-- architecture.md
|   |-- window-behavior.md
|   |-- content-model.md
|   `-- testing.md
|-- public/
|   |-- icons/
|   |-- wallpapers/
|   `-- images/
|       |-- profile/
|       `-- projects/
|-- src/
|   |-- app/
|   |   |-- (desktop)/
|   |   |   |-- layout.tsx
|   |   |   |-- page.tsx
|   |   |   `-- [appId]/page.tsx
|   |   |-- api/
|   |   |-- global-error.tsx
|   |   |-- globals.css
|   |   |-- layout.tsx
|   |   `-- not-found.tsx
|   |-- apps/
|   |   |-- about/
|   |   |-- projects/
|   |   |-- experience/
|   |   `-- settings/
|   |-- os/
|   |   |-- app-registry/
|   |   |-- desktop/
|   |   |-- onboarding/
|   |   |-- preferences/
|   |   |-- responsive/
|   |   |-- routing/
|   |   |-- taskbar/
|   |   `-- window-manager/
|   |-- server/
|   |   |-- actions/
|   |   |-- content/
|   |   |-- env/
|   |   `-- integrations/
|   |       |-- recommendations/
|   |       `-- watchlist/
|   |-- components/
|   |   |-- shared/
|   |   `-- ui/
|   |-- assets/fonts/
|   |-- hooks/
|   |-- lib/
|   |-- styles/
|   `-- test/
`-- e2e/
```

Module responsibilities

- src/os contains operating-system mechanics and presentation.
- src/apps contains portfolio applications running inside the OS.
- src/server contains secret-bearing integrations, environment validation, server actions, and content loaders.
- src/components/ui contains foundational primitives only.
- src/components/shared contains genuinely cross-application presentation components.
- Feature-specific components stay beside their owner.
- src/lib remains small and generic; it is not a dumping ground.

Provider composition

```
IshpreetOSProviders
|-- PreferencesProvider
|-- ResponsiveModeProvider
|-- DesktopSessionProvider
`-- RoutingCoordinator
```

The wrapper is convenient, but the providers remain independent. A clock-format change must not run the window reducer; a focus change must not rewrite preferences.

Import rules

- Keep the existing @/* path alias.
- Use named exports for normal modules; use default exports only where Next.js requires them.
- Avoid giant barrel files. A small index.ts is allowed only as an intentional public module boundary.
- Applications may call the OS command API but may not import reducer internals.
- Client code may not import from @/server.
- Applications may not import directly from one another; promote truly shared code deliberately.

Styling ownership

- tokens.css owns reusable design values.
- base.css owns document-level defaults.
- typography.css owns font roles and readable defaults.
- motion.css owns shared motion variables and reduced-motion rules.
- globals.css imports the system and Tailwind.
- Tailwind handles normal component layout; narrowly scoped CSS Modules handle complex geometry or effects.
- Do not introduce CSS-in-JS or repeated arbitrary hex values in applications.

17. Testing and Quality Strategy

Test placement

- Unit and component tests live beside the module they verify.
- Shared factories, fixtures, and setup stay in src/test.
- Full browser journeys stay in the root e2e directory.
- Every phase includes its tests; a later testing phase does not excuse missing earlier coverage.

Unit-test priorities

- Reducer branches and invalid transitions.
- Taskbar and stacking invariants.
- Cascade, wrapping, clamping, and mode-specific geometry.
- Responsive-mode selection.
- Registry and route lookup.
- Content, environment, form, settings, and provider schemas.
- Integration mapping and failure results.

Component-test priorities

- Window controls and focus-and-activate behavior.
- Taskbar switching and Show Desktop.
- Settings persistence and reset separation.
- Project link variants and status semantics.
- Recommendation form states.
- Watchlist success, empty, and unavailable states.

Playwright journeys

- Fresh root visit with boot and welcome.
- Returning root visit without boot.
- Direct route that bypasses onboarding.
- Open, focus, minimize, restore, maximize, close, and reopen.
- Home/Show Desktop plus browser Back.
- Mobile one-visible-app switching.
- Tablet touch dragging.
- Desktop resizing and bounds.
- Refresh wipes session but preserves settings.
- Protected recommendation success and failure paths with mocked providers.

Coverage philosophy

Do not chase an arbitrary whole-project percentage. Require complete branch coverage for reducer and geometry utilities because they define core invariants. Judge UI coverage through critical scenarios and regression value.

Universal phase gate

- The phase objective is demonstrably complete.
- Relevant automated tests pass.
- Lint, type checking, and production build pass.
- Required manual device checks pass.
- Documentation reflects the implemented behavior.
- No unused dependency or abandoned experiment remains.
- No unresolved critical or high-severity defect is carried into the next phase.

18. Implementation Sequence

Phase	Name	Outcome
0	Guardrails	Repository architecture, documentation, CI, environment validation, starter cleanup, and initial content dependencies.
1	Contracts	Application registry, loader map, Zod schemas, Markdown/YAML loaders, environment contracts, and representative fixtures.
2	OS Core	Pure reducer, commands, selectors, taskbar/stack rules, geometry, cascade, bounds, and mode transitions.
3	Desktop Shell	Client measurement, responsive controller, providers, persistence, fonts/tokens, wallpaper, icon grid, taskbar, and placeholder windows.
4	Routing and Onboarding	Dynamic application route, metadata, history coordination, direct links, Home behavior, boot, welcome, and 404 recovery.
5	Window Interaction	react-rnd spike, WindowInteractionLayer, desktop drag/resize, tablet touch drag, bounds, and real-device validation.
6	Visual System	Final window treatments, typography, accents, focus veil, wallpaper selection, Motion, and reduced-motion behavior.
7	Applications	About first, then Projects, Experience, and Settings/Privacy. External integration areas remain placeholders.
8	Integrations	Notion watchlist plus protected Turnstile/Upstash/Resend recommendation delivery.
9	Shipping Surface	SEO, social previews, analytics, recovery, licences, privacy, CSP, and security headers.
10	Release	Cross-device QA, browser matrix, performance, bundle audit, dependency cleanup, production deployment, and smoke test.

Application order

About may be the first fully designed application because it is the area Ishpreet is most excited to build. That choice does not pull its integrations forward: Notion and recommendation delivery remain Phase 8. Projects and Experience follow; Settings functionality is introduced as needed for foundation testing and completed during the application phase.

Content parallel track

- Biography and social links.
- Experience entries.
- Project descriptions and screenshots.
- Wallpaper and personal photography selection.
- Icon-family and font evaluation.
- Asset licence bookkeeping.

Structural placeholders are allowed through the foundation and OS phases. No placeholder may survive the completion gate of its application or the final release.

Timeline policy

There is no fixed launch date. A realistic timeline will be estimated only after the responsive shell and react-rnd prototype are complete, because they represent the main technical uncertainty. Quality and phase confidence take priority over forcing a one-month schedule.

19. Development Workflow and Ownership

Learning-first ownership

Owner	Primary work
Ishpreet leads	Portfolio content, application interiors, project cards, experience timeline, settings controls, visual styling, straightforward component tests, and later Zod schemas with guidance
Build together	Registry, content loaders, reducer, actions, responsive controller, preferences, routing behavior, lifecycle, and Playwright journeys
AI leads or scaffolds	Geometry edge cases, WindowInteractionLayer, route synchronization, server-only integration boundaries, protected email flow, CI, and production hardening

This split is flexible. Ishpreet remains the product owner and may take over more code for learning or delegate a complex component when it would otherwise stall the project.

Branch workflow

- Use short-lived feature branches with one coherent purpose.
- Include tests and documentation in the same branch as the behavior.
- Use pull requests and Vercel previews even as a solo developer.
- Merge a dependency phase before beginning work that deeply relies on it.
- Do not use a month-long development branch.
- Do not mark a Notion task In Progress until implementation actually begins.
- Do not mark a task Done until its acceptance criteria and phase gate pass.

Planning and execution sources

- This handbook records the approved product and implementation baseline.
- Repository docs evolve with implemented architecture.
- The Notion board records status, phase, build order, priority, release, type, acceptance criteria, and dependencies.
- Code and tests are the final evidence of implemented behavior.

20. v1 Release Gate

- All four core applications are complete with final content.
- Routing, direct links, browser history, and refresh behavior are complete.
- Mobile, narrow tablet, spacious tablet, and desktop window behavior is verified.
- Boot, welcome, direct-entry bypass, replay, and onboarding persistence are verified.
- Settings persistence, privacy, and reset separation are complete.
- Local content schemas and build-time validation pass.
- Notion watchlist succeeds and fails gracefully.
- Recommendation form is protected and sends only from production to the real inbox.
- Metadata, canonical URLs, sitemap, robots rules, and Open Graph images are complete.
- Credits and asset licensing are complete.
- Approved anonymous analytics and Speed Insights are operating.
- Security headers and Content Security Policy are verified in production.
- Critical unit, component, and Playwright tests pass.
- Supported browsers and physical devices pass manual smoke tests.
- Performance targets are met and no external provider blocks desktop loading.
- No known critical or high-severity defect remains.
- No v1.1 package, unfinished placeholder, or abandoned dependency ships.

Release principle

A page that looks finished is not automatically finished. A phase or feature is complete only when its tests, documentation, failure behavior, and relevant device checks are also complete.

21. Non-negotiable Invariants

1. One application instance per AppId.
2. Opening an existing app restores/focuses it; it never creates a second instance.
3. Taskbar order is initial-open order and does not change on focus.
4. Close removes the app; reopen appends it.
5. Stacking order is independent from taskbar order.
6. No window edge may leave the usable desktop.
7. Mobile and narrow tablet show one fixed, non-movable, non-resizable window.
8. Spacious tablet supports multiple draggable windows without resizing.
9. Desktop supports multiple draggable and resizable windows.
10. Maximize fills the usable IshpreetOS desktop and keeps the taskbar visible.
11. Minimized and hidden apps stay mounted; closed apps unmount.
12. Window geometry is session-only and mode-specific.
13. Preferences persist locally per browser and never change the site for other visitors.
14. Refresh resets the desktop session but preserves preferences and onboarding.
15. Direct application routes bypass boot and welcome.
16. Focus does not add browser-history noise.
17. External services cannot block or crash the desktop.
18. Raw HTML is disabled in Markdown.
19. Application body text is pure black on cream.
20. No automatic sound.
21. No visitor location request.
22. No personal form content in analytics or logs.
23. No v1.1 dependencies in v1.

22. Prototype and Validation Register

Item	Approved direction	What remains to validate
Responsive thresholds	Four capability-based modes	Exact width/height and pointer thresholds on target devices
Default window size	Shared percentage-based geometry	Exact proportions and cascade offsets in prototypes
react-rnd	Preferred interaction base behind wrapper	Touch, bounds, mode transitions, performance, and controlled-state behavior
Motion	Add after stable behavior	Exact duration/easing and whether every planned transition earns its cost
Serif font	Open-source Times-like local font	Exact family after visual and readability comparison
Pixel font	Readable local pixel-style font	Exact family and small-size legibility
Icon pack	One coherent publicly licensed pixel family	Exact pack and coverage for all apps
Accent values	Blue, green, purple, orange, pink	Exact tokens and contrast across wallpapers
Technology tags	Allowed if visually useful	Density and wrapping inside resized project windows
Reactive About heading	Heading/short phrase only	Performance, pointer behavior, and reduced-motion alternative
Spotify design	Server adapter in v1.1	Revisit interface and data presentation when implementation starts

23. Operating Instructions for Future AI Collaborators

Before changing code

1. Read this handbook and the repository architecture documents.
2. Inspect the current Notion task and its acceptance criteria.
3. Inspect the repository rather than assuming the starter structure still applies.
4. Identify the current responsive mode, lifecycle, routing, and persistence implications of the change.
5. Check whether the requested work belongs to v1 or v1.1.
6. Check the dependency policy before installing a package.

While implementing

- Preserve separation among routes, OS domain, React providers, application components, server integrations, and local content.
- Keep core state transitions and geometry pure and testable.
- Use project-owned wrappers for replaceable third-party behavior.
- Add tests with the behavior, not afterward.
- Do not silently broaden scope because an API or component makes an extra feature easy.
- Do not write secrets, private email addresses, or provider tokens into source, tests, analytics, or documentation.
- Keep placeholders explicit and remove them before the relevant completion gate.

When a decision appears inconsistent

- Prefer the newest explicit decision over old Notion wording or starter code.
- Preserve the user's intent, not accidental phrasing or typos.
- Explain the concrete conflict and its user or technical impact.
- Recommend a resolution and ask Ishpreet when changing the product model would be required.
- Do not treat implementation difficulty as authorization to remove a feature.

When finishing a task

- Run the task's automated and manual checks.
- Update repository documentation for material architecture behavior.
- Update the Notion status only when the documented acceptance criteria pass.
- Report remaining risks, deferred work, and any dependency added or removed.
- Do not declare a phase complete while critical defects, missing tests, or missing documentation remain.

Appendix A. Window Transition Reference

Current state	Command	Result
Closed	Open	Create one instance, append taskbar, initialize target-mode geometry, focus
Open and visible	Activate desktop icon	Focus existing instance; no reset or duplicate
Open and behind	Click window/control	Raise and activate clicked control in one action
Open and visible	Minimize	Hide, preserve component, retain taskbar position
Minimized	Activate taskbar	Restore and focus
Normal	Maximize	Remember normal geometry and fill usable desktop
Maximized	Restore	Return to valid remembered normal geometry
Any open	Close	Unmount, remove from orders, destroy transient state
Any mix	Show Desktop	Minimize every open app and route to root
Mobile visible A	Activate open B	Hide A, show/focus B, preserve both
Any	Viewport mode change	Select/recalculate target-mode geometry without overwriting other modes
Any	Refresh	Clear session; rebuild from current route; preserve preferences/onboarding

Appendix B. Notion Build Order

Order	Phase	Task
1	0	Foundation: repository architecture, documentation, and CI
2	1	Foundation: application registry and content contracts
3	1	Content: final portfolio copy and licensed assets
4	2	Core: window state reducer and session rules
5	2	Core: geometry, cascade, bounds, and mode transitions
6	3	Shell: responsive modes and client initialization
7	3	Shell: providers, preferences, and persistence
8	3	Shell: desktop, icon grid, and taskbar
9	4	Routing: app routes, browser history, and direct-link entry
10	4	Onboarding: first-visit boot sequence
11	4	Onboarding: welcome terminal window
12	5	Windows: WindowFrame and interaction adapter
13	5	Test: window manager and responsive interactions
14	6	Design: visual system and motion
15	7	App: About
16	7	App: Projects
17	7	App: Experience
18	7	App: Settings and Privacy
19	8	Integration: Notion watchlist
20	8	Integration: protected recommendation submission
21	9	Shipping: SEO, recovery, sharing, and analytics
22	9	Shipping: licensing, privacy, and security headers
23	10	Release: cross-device QA and production deployment
101	v1.1	Spotify music widget
102	v1.1	Toronto weather widget
103	v1.1	Terminal Easter eggs

Appendix C. Technical Reference Links

Next.js forms and Server Actions
https://nextjs.org/docs/app/guides/forms

Next.js Route Handlers
https://nextjs.org/docs/app/getting-started/route-handlers

Next.js server fetch caching
https://nextjs.org/docs/app/api-reference/functions/fetch

react-rnd
https://github.com/bokuweb/react-rnd

Motion for React
https://motion.dev/docs/react

Zod
https://zod.dev/

Notion integrations
https://www.notion.com/help/create-integrations-with-the-notion-api

Resend with Next.js
https://resend.com/nextjs

Upstash rate limiting
https://upstash.com/docs/redis/sdks/ratelimit-ts/overview

Cloudflare Turnstile
https://developers.cloudflare.com/turnstile/get-started/

Spotify authorization
https://developer.spotify.com/documentation/web-api/concepts/authorization

Open-Meteo
https://open-meteo.com/

Vercel Web Analytics
https://vercel.com/docs/analytics

Vercel Speed Insights
https://vercel.com/docs/speed-insights

Vercel guidance on Cloudflare proxying
https://vercel.com/kb/guide/cloudflare-with-vercel

Appendix D. Glossary

Term	Definition
Usable desktop	Viewport area remaining after reserving the IshpreetOS taskbar and any required safe areas
AppId	Stable registry identifier for one application type
Desktop session	Transient open-window, taskbar, stack, and geometry state that resets on refresh
Preference	Per-browser local setting that survives refresh and return visits
Geometry	A window's position and, where applicable, size within a specific responsive mode
Focus	The application that receives interaction and appears at the top of the stack
Meaningful opening	A route-worthy app navigation, unlike a simple z-order focus change
Direct entry	A first page load on an application route instead of the root desktop
Capability-first integration	Internal service interface organized around a need such as watchlist or recommendations, not around a vendor
Phase gate	Tests, documentation, manual checks, and defect threshold required before dependent work begins

Final instruction

Build IshpreetOS carefully enough that its complexity feels intentional to visitors and understandable to its owner. The operating-system illusion should come from consistent rules, not from visual imitation alone.